

Sistemas de Big Data

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



**Sistemas de
Big_Data**

Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Capítulo 10

Inteligencia artificial en el análisis de datos

José Ángel Martín Baos

En el capítulo anterior se realizó una breve introducción al framework de programación Apache Spark, el cual es utilizado por un gran número de empresas para realizar análisis de datos en el contexto de sistemas Big Data. Posteriormente, estudiamos las principales técnicas de análisis de datos empleadas por los científicos de datos de manera tradicional para extraer información de valor para la organización. En este capítulo complementaremos las técnicas anteriores y estudiaremos las principales técnicas de inteligencia artificial y aprendizaje automático aplicadas en el análisis de datos.

Es importante comprender que los algoritmos de inteligencia artificial requieren de datos para poder construir su inteligencia, no solo en el momento inicial, sino también a posteriori para poder mejorar su aprendizaje conforme va pasando el tiempo. Es evidente que cuanto más grande sea el conjunto de datos que estos algoritmos utilizan para realizar su entrenamiento, más precisos y valiosos serán los resultados obtenidos. No obstante, es importante recordar que también se da la relación opuesta, es decir, cantidades tan grandes de datos no tendrían el valor suficiente si no contamos con buenos modelos de inteligencia artificial que puedan transformar estos datos en información, y por lo tanto en valor.

A día de hoy, la totalidad de las empresas más importantes del mundo utilizan esta combinación de Big Data y aprendizaje automático para obtener información e indicadores valiosos para su negocio. Pongamos por ejemplo el caso de la famosa tienda online de Amazon. Esta plataforma recolecta cantidades ingentes de datos de los clientes y los visitantes de su sitio web. Posteriormente, recurren a técnicas de inteligencia artificial para extraer conocimiento de estos datos y poder así proporcionar mejores anuncios a sus clientes, tomar decisiones de negocio que aporten gran valor o incluso crear sus propios productos en base a las predicciones obtenidas (véase la marca AmazonBasics).

10.1. Detección de outliers

En análisis de datos, un *outlier* (también conocido como **valor atípico**) es una observación distante o muy diferente de otras observaciones en una colección de datos. Por lo tanto, son observaciones con valores inusuales, ya sean muy grandes o muy pequeños en relación al resto de la muestra. La figura 10.1 muestra un ejemplo de un outlier en una muestra de datos en dos dimensiones.

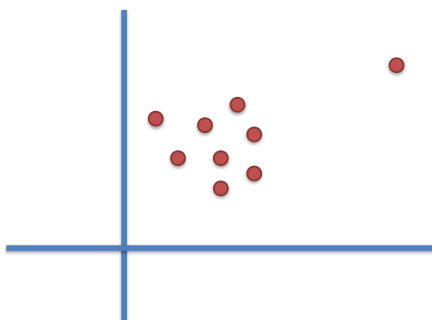


Figura 10.1: Ejemplo de un outlier en una muestra de datos en dos dimensiones.

Una vez se han identificado los outliers, los métodos habitualmente utilizados para tratar estos valores atípicos pueden variar desde simplemente eliminarlos o trasladarlos a otro conjunto de datos separado o bien sustituir estos outliers por otros valores más razonables o adecuados (como por ejemplo la media, la moda, un valor fijo, etc). El lector puede pensar que esta forma de procesar los valores atípicos quizá no sea un proceso tan complicado, pero es algo que hay que pensar y repensar seriamente antes de introducir cualquier proceso para identificar y tratar los valores atípicos en grandes conjuntos de datos, ya que la carga computacional será muy elevada.

Otra consideración a tener en cuenta es la siguiente: ¿la aparición de outliers en nuestro conjunto de datos es un indicador de que tenemos datos que son defectuosos o erróneos (por ejemplo, fallo en un sensor) o por el contrario están recogiendo valores atípicos casados por observaciones con características nuevas e interesantes dentro del conjunto de datos (por ejemplo, una situación anómala en una cadena de producción)? En cualquier caso, la presencia de outliers en los datos requerirán de un método válido y, especialmente en el contexto del Big Data, robusto para tratarlos.

Los métodos más empleados para determinar los outliers se basan en construir bandas (intervalos) en los que deben estar los datos. Si un dato se sale de dicha banda se dictamina que se trata de un outlier. Formalmente,

$$\text{Si } x_i \notin [x_L, x_U] \Rightarrow x_i \text{ es un outlier} \quad (10.1)$$

Uno de los métodos más utilizados para determinar estas bandas es el **rango intercuartiles**, que se obtiene mediante la expresión:

$$x_L = Q_1 - k(Q_3 - Q_1) \quad (10.2)$$

$$x_U = Q_3 + k(Q_3 - Q_1) \quad (10.3)$$

donde Q_1 y Q_3 son el primer y tercer cuartil de la muestra, respectivamente. Los valores usuales para k son $k = 1,5$ (outlier leve) y $k = 3$ (outlier extremo).

10.2. Reducción de la dimensionalidad

En Big Data, muchos de los problemas de aprendizaje automático implican conjuntos de datos que contienen, además de una gran cantidad de observaciones, miles o incluso millones de características para cada observación. Esto no sólo hace que el entrenamiento de los algoritmos de aprendizaje automático sea extremadamente lento, sino que también puede hacer que sea mucho más difícil encontrar una buena solución. Este es un problema, muy famoso en el mundo del aprendizaje automático, se conoce a menudo como la **maldición de la dimensionalidad**. Afortunadamente, en los problemas del mundo real, a menudo es posible reducir el número de características de manera considerable, permitiendo que un problema intratable se convierta en uno manejable.



Pérdida de información. Nótese que la reducción de la dimensionalidad de un conjunto de datos, pese que aceleraría la ejecución de los algoritmos de aprendizaje automático, conllevaría a un rendimiento ligeramente peor, ya que se produciría una pérdida de información. El lector puede buscar la analogía con la compresión de una imagen o un vídeo a formatos de menor tamaño, pero también una calidad ligeramente más baja.

10.2.1. La maldición de la dimensionalidad

Algunos métodos de aprendizaje automático requieren del cálculo de la distancia entre cada par de observaciones. Supongamos que un determinado objeto viene representado por el vector $\mathbf{x} = (x_1, \dots, x_k)$ y otro objeto por el vector $\mathbf{y} = (y_1, \dots, y_k)$, dónde k es el número de dimensiones. Si quisiéramos calcular la distancia entre ambos objetos aplicando la distancia euclídea tendríamos que calcular la expresión:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^k (x_i - y_i)^2 \quad (10.4)$$

El número de operaciones que tendríamos que realizar es $k - 1$ sumas, k restas y k potencias, por tanto la operación para calcular la distancia entre dos objetos tiene una complejidad computacional de orden $\mathcal{O}(k)$. Si el número de variables k del problema fuera muy elevada el coste computacional pudiera a llegar a ser muy elevado. Por ejemplo, si tuviésemos que calcular la distancia entre las n observaciones en un conjunto de datos entonces tendríamos que aplicar $\frac{(n-1)n}{2}$ la fórmula anterior. En este ejemplo el coste computacional puede llegar a ser prohibitivo si los valores de n y k son elevados.

Otros métodos de aprendizaje automático requieren realizar cálculos del tipo de multiplicar una matriz de datos X por un vector de parámetros α . Por ejemplo supongamos que tenemos tres datos $\mathbf{x}_1 = (1, 0, 3, 4)$, $\mathbf{x}_2 = (-1, 0, 2, 1)$ y $\mathbf{x}_3 = (1, 1, 0, -1)$, construimos la matriz (tabla) de datos

$$X = \begin{bmatrix} 1 & 0 & 3 & 4 \\ -1 & 0 & 2 & 1 \\ 1 & 1 & 0 & -1 \end{bmatrix} \quad (10.5)$$

El cálculo matricial se expresa por:

$$X \cdot \alpha = \begin{bmatrix} 1 & 0 & 3 & 4 \\ -1 & 0 & 2 & 1 \\ 1 & 1 & 0 & -1 \end{bmatrix} \cdot \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \end{bmatrix} = \begin{bmatrix} \alpha_1 + 3\alpha_3 + 4\alpha_4 \\ -\alpha_1 + 2\alpha_3 + \alpha_4 \\ \alpha_1 + \alpha_2 - \alpha_4 \end{bmatrix} \quad (10.6)$$

La matriz X tiene $n = 3$ filas y $k = 4$ columnas. El número de multiplicaciones realizadas es $n \cdot k$ y el número de sumas $n(k - 1)$. Por tanto el número de dimensiones de los datos afecta linealmente cada vez que se tenga que realizar una multiplicación.

Observamos pues que el número de dimensiones de un dataset juega un papel importante en la complejidad de los algoritmos de aprendizaje automáticos utilizados. A continuación, estudiaremos uno de los mecanismos más comúnmente empleados para reducir la dimensionalidad de un problema.

10.2.2. Análisis de componentes principales

En la mayoría de los problemas del mundo real, los datos de entrenamiento no están repartidos uniformemente en todas las dimensiones del problema. Muchas características son casi constantes, mientras que otras están muy correlacionadas (recordar el concepto de correlación estudiado en el tema anterior). Como resultado, todas las observaciones se encuentran en realidad en un subespacio de dimensiones mucho más bajas del espacio de altas dimensiones original. El concepto anterior es muy abstracto, por lo tanto vamos a simplificarlo con un ejemplo. La figura 10.2 muestra un conjunto de datos en dos dimensiones, sin embargo, nótese que todas las observaciones se encuentran cerca de una recta (representada en color negro). Por lo tanto, si proyectamos cada elemento perpendicularmente en este subespacio

(lo cual se ha representado usando las líneas cortas moradas que conectan las observaciones con la recta negra), obtenemos un nuevo conjunto de datos con una única dimensión que se muestra en la figura 10.3. Hemos, por tanto, reducido el número de dimensiones pero manteniendo la mayor variabilidad de los datos posible.

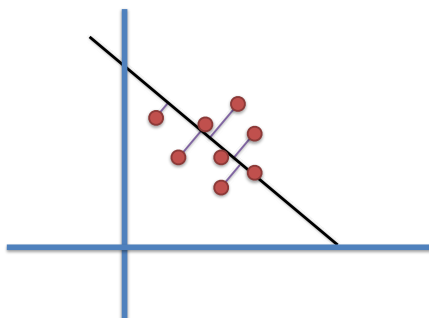


Figura 10.2: Proyección de un conjunto de datos en dos dimensiones sobre la recta en la que preservan mayor varianza.



Figura 10.3: Resultado de proyectar los datos de la figura 10.2 sobre la recta en la que preservan la mayor varianza.

El **Análisis de Componentes Principales** (o PCA por sus siglas en inglés) es, con diferencia, el algoritmo de reducción de la dimensionalidad más popular. Se basa en identificar el hiperplano más cercano a los datos (el que preserve más su variabilidad) y, a continuación, proyecta los datos sobre él, como se ha visto en la figura 10.2.

La aplicación del algoritmo PCA como parte del preprocesamiento de los datos tiene dos finalidades: Por un lado permite reducir el número de dimensiones del conjunto de datos, como se ha mostrado en el ejemplo anterior; Por otro lado, dado que el PCA se calcula encontrando los componentes que explican la mayor cantidad de varianza, ayuda a reducir el ruido de los datos.

Además, el PCA también se puede utilizar para otros casos de uso, como por ejemplo, proyectar datos multidimensionales en espacios de dos o tres dimensiones (que pueden visualizarse fácilmente). El PCA también se puede emplear para comprimir la información con el fin de almacenarla y transmitirla de forma más eficiente. Por ejemplo, puede utilizarse para comprimir imágenes sin perder demasiada calidad, o en el procesamiento de señales.

Veamos ahora las ventajas y las desventajas en el uso del algoritmo PCA. Las principales ventajas son:

- **Fácil de realizar.** El PCA se basa en métodos básicos del álgebra lineal. Estas técnicas están ampliamente difundidas y se recogen en el software dedicado al aprendizaje automático.
- **Acelera algoritmos de aprendizaje automático.** Los algoritmos de aprendizaje automático convergen más rápidamente cuando se entrenan con componentes principales en lugar de los datos originales. Esto es debido fundamentalmente a que el número de variables del problema de entrenamiento se reduce.
- **Palía algunos de los problemas de la alta dimensionalidad de los datos.** La alta dimensionalidad de los datos hace que los algoritmos basados en la regresión se sobreajusten fácilmente. Al utilizar el PCA se elimina ruido de los datos y se reduce la dimensión del conjunto de datos de entrenamiento, evitando así el sobreajuste de los modelos de regresión.

No obstante, el PCA presenta algunas desventajas, como por ejemplo:

- **Baja interpretabilidad de los componentes principales.** Los componentes principales son combinaciones lineales de las características de los datos originales, por lo tanto no es fácil de interpretar. Por ejemplo, después de calcular los componentes principales es difícil saber cuáles son las variables más importantes del conjunto de datos original.
- **Equilibrio entre la pérdida de información y la reducción de la dimensionalidad.** La pérdida de información es un peaje que hay que pagar por emplear el PCA. El modelador debe establecer un equilibrio entre la reducción de la dimensionalidad y la pérdida de información.

10.3. Aprendizaje no supervisado

El aprendizaje **no supervisado** parte de datos no etiquetados previamente. Por lo tanto, la función de estos algoritmos es el de agrupar los datos en función de su similitud, de forma que se puedan crear grupos sin necesidad de definir cómo es cada uno de los integrantes del grupo.

10.3.1. Clustering

El principal tipo de problemas de aprendizaje no supervisado son los denominados problemas de **clustering**, el cual consiste en buscar patrones dentro de los datos. Esto es, encontrar subconjuntos de observaciones que son similares entre sí. Por ejemplo, puede utilizarse para buscar grupos de clientes basados en su comportamiento, productos similares en base a sus características, tipos de películas en base a su audiencia, etc. Si se realiza un análisis cluster sobre los clientes de una determinada empresa se puede encontrar patrones de clientes y con ellos ofrecer productos especialmente diseñados para estos segmentos.

Matemáticamente se formula como un problema en el cual se debe determinar las etiquetas y de los datos, de modo que si dos observaciones i y j son similares le asociemos la misma etiqueta, $y_i = y_j$. Una dificultad de este problema es que no se parte de un conjunto previo de etiquetas del cual aprender.

Hay una gran variedad de algoritmos de clustering entre los que elegir y no hay un único algoritmo de clustering que sea el mejor para todos los casos. En su lugar, es una buena idea explorar una amplia gama de algoritmos de clustering así como diferentes configuraciones para cada algoritmo y seleccionar el que mejor se adapte a nuestro problema. En la siguiente sección introduciremos el que, posiblemente, sea el algoritmo de clustering más famoso, así como uno de los más empleados en la actualidad, el algoritmo de las K -medias.

10.3.2. Algoritmo de las K -medias

El algoritmo de las K -medias (denominado K -means en inglés) es un algoritmo iterativo que trata de dividir un conjunto de datos en K subgrupos distintos (denominados *clusters*) sin solapamiento, donde cada elemento del conjunto de datos pertenece únicamente a un grupo. Este algoritmo intenta que los elementos pertenecientes a cada cluster sean lo más parecidos posibles entre ellos, pero a la misma vez trata que los distintos clusters sean lo más diferentes posible (es decir, se encuentren alejados). Para ello, asigna cada una de las observaciones de un conjunto de datos a cada uno de los clusters definidos, de forma que la suma de la distancia al cuadrado entre los distintos elementos del cluster y su **centroide** (definido como la media aritmética de todos los elementos que pertenecen a ese cluster) sea mínima. Cuanto menor sea la variación dentro de cada uno de los clusters, más homogéneos (similares) serán los elementos dentro del mismo cluster.

El algoritmo de las K -medias sigue los siguientes pasos:

0. **Inicialización.** Se eligen de manera aleatoria K objetos (individuos) del conjunto de datos como los centroides iniciales para los K grupos.
1. **Agrupación.** Se calcula la distancia entre cada objeto y cada centroide. Después se asigna cada objeto al grupo k para el cual la distancia entre el objeto y su centroide sea menor.
2. **Actualización de centroides.** Se recalculan los K centroides anteriores tomando la media de todos los objetos asignados a cada centroide en el paso anterior.
3. **Criterio de paro.** Se comprueba si en esta iteración ha habido algún cambio en los centroides. De ser este el caso, se vuelve al paso 1 (Agrupación) y se ejecuta una nueva iteración. En caso contrario, el algoritmo termina. Está garantizado que el algoritmo converge en un número finito de pasos (normalmente bastante pequeño), es decir, no oscilará eternamente.



Ejemplo visual de las K -medias. En el siguiente  Enlace: <http://www.naftaliharris.com/blog/visualizing-k-means-clustering/> se puede visualizar el funcionamiento del algoritmo K -means para diferentes conjuntos de datos, mostrando que es adecuado para problemas en los que los grupos no estén engarzados uno dentro de otros.

La figura 10.4 muestra el resultado de ejecutar el algoritmo de las K -medias sobre un conjunto de datos determinado, el cual contiene tres grupos de datos. Como se puede ver, una vez fijado $K = 3$, el algoritmo es capaz de determinar los distintos grupos. Las estrellas negras representan los centroides tras ejecutar el algoritmo. Cada observación se ha representado utilizando distintos colores en función del grupo asignado (recuerda, se asigna a aquel grupo para el cual distancia entre el objeto y su centroide sea menor).

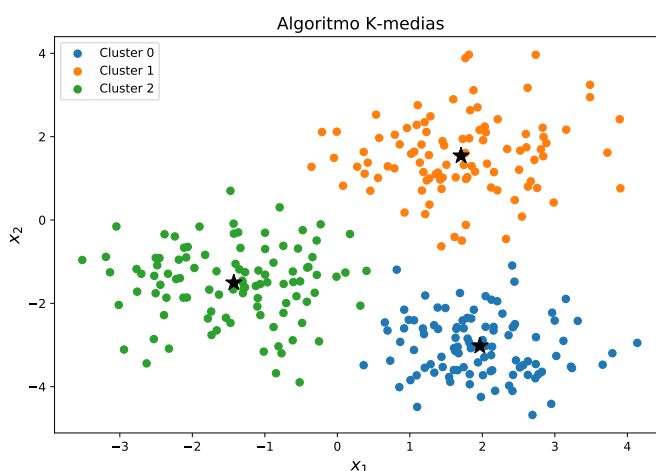


Figura 10.4: Ejemplo de la ejecución del algoritmo de las K -medias.

Uno de los problemas más importantes presentes en el algoritmo de las K -medias es que este converge a mínimos locales. Un **mínimo local** es el valor de una función, que es menor que los valores de la función en puntos cercanos, pero que no es el menor de todos los valores. Muchos de los algoritmos empleados en análisis cluster exhiben esta problemática ya que estos resuelven un problema de optimización discreto. La figura 10.5 ilustra el concepto de mínimo local en un problema de optimización continua. La solución $\underline{x}$ es un mínimo local ya que si introdujésemos pequeños desplazamientos suyos en un cierto entorno o vecindad se incrementaría el valor de la suma de errores al cuadrado (SSE). La solución $\underline{x}^*$ es el óptimo global porque tiene la imagen menor. Notar que $\underline{x}^*$ también es un mínimo local.

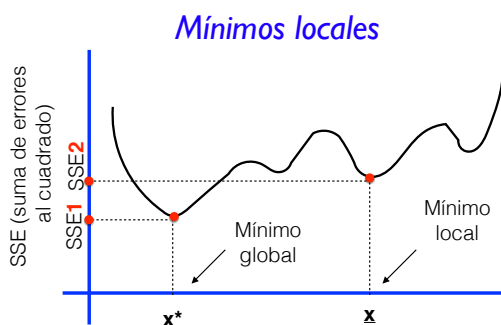


Figura 10.5: Mínimos locales.

La elección de los valores iniciales de los centroides del algoritmo de las K -medias es crucial para evitar los numerosos óptimos locales del análisis cluster. Se han propuesto numerosas técnicas para determinar estos valores, que pasan desde tomar los K primeros datos de la base de datos, elegirlos aleatoriamente, incluso basados en la experiencia anterior de un experto. La mayoría de las implementaciones actuales eligen los valores iniciales de los centroides de manera aleatoria. Por este motivo, es importante repetir el algoritmo completo un número determinado de veces (variando la inicialización de los centroides) y elegir la mejor solución. En Spark el número de inicializaciones se controla mediante el parámetro `initSteps`.

10.3.3. Otros algoritmos de clustering

Además del algoritmo de las K -medias, hay otra serie de algoritmos de clustering que son muy utilizados en la actualidad. A continuación vamos a describir brevemente los más relevantes.

1. **Fuzzy c -means.** Es una clase de algoritmos de clustering similar al algoritmo de las K -medias pero donde cada elemento tiene un grado de pertenencia difuso a los distintos grupos. Es decir, mientras que en el K -medias cada elemento solo puede pertenecer a un grupo, en el algoritmo fuzzy c -means cada elemento tiene asignado un grado de pertenencia a cada uno de los c grupos definidos.
2. **Algoritmos jerárquicos.** Son una clase de algoritmos que agrupa los datos basándose en la distancia entre cada uno de estos datos, buscando que los datos dentro de cada cluster sean lo más similares entre sí. Estos pueden ser de dos tipos: i) **aglomerativos** en los que inicialmente cada objeto es su propio grupo y un par de estos grupos se van juntando en cada iteración hasta cierto punto y ii) **divisivos** en los que se parte de un solo grupo y en cada iteración se divide uno de los grupos existentes hasta cierto punto. No obstante, este tipo de algoritmos son muy costosos para grandes cantidades de datos, por lo que no se suelen utilizar en contextos de Big Data.

3. **Algoritmos basados en el concepto de densidad.** Estos se basan en la suposición de que los grupos están definidos en áreas de gran concentración (densidad) de datos, mientras que las zonas poco densas separan los grupos. Dentro de este tipo de algoritmos destacamos: el algoritmo **DBSCAN** y el **algoritmo basado en picos de densidad**, los cuales son bastante eficientes y funcionan relativamente bien para gran cantidad de datos.

10.4. Aprendizaje supervisado

El **aprendizaje supervisado** supone que partimos de un conjunto de datos etiquetado previamente. Un algoritmo de aprendizaje supervisado realizará su aprendizaje mediante el uso tanto las características de cada observación como una etiqueta que tendrá que aprender. Los algoritmos de aprendizaje supervisado pueden dividirse en:

- **Problemas de clasificación.** En este tipo de problemas se debe predecir el valor de la variable cualitativa y a partir de la variable x . En clasificación la variable x se denomina **características** o **atributos** mientras que y se denomina **etiqueta**.

Un ejemplo de este tipo de problemas es el desarrollo de técnicas de aprendizaje automático para coches autónomos. El coche autónomo se basa en tres pilares: el internet de las cosas (IoT), técnicas de aprendizaje automático aplicado a Big Data y la conexión a internet a tiempo real. En la construcción de *los ojos* del vehículo se requiere resolver el problema de clasificar las imágenes que rodean al vehículo y poder así determinar que tipo de objeto está delante, detrás o a los lados del vehículo.

- **Problemas de regresión.** En este tipo de problemas se predice el valor de una variable continua y conocida el valor de la variable x . En este problema la variable x se suele denominar **regresor** o **variable explicativa** mientras que y se denomina **variable respuesta** o **variable dependiente**.

Un ejemplo de este tipo de problema aparece en los sistemas de recomendación que emplean plataformas de streaming como Spotify, YouTube y Netflix. Estos sistemas estiman funciones de regresión $y = f(x_1, x_2)$ que permiten estimar el tiempo de consumo de un usuario (y) para un usuario que ha consumido el producto 1 con características x_1 e inicia el consumo de un segundo producto 2 con características x_2 .

A continuación, presentaremos los principales tipos de algoritmos de aprendizaje supervisado utilizados para realizar análisis de datos en el contexto del Big Data.

10.4.1. Regresión logística

La **regresión logística** es similar a la regresión lineal (estudiada en el capítulo anterior) pero, a diferencia de la última que permite predecir valores continuos en función de unos valores de entrada, la regresión logística predice si algo pertenece a una clase o no. Para ello, la regresión logística calcula las probabilidades de pertenencia a una clase, utilizando tanto atributos discretos como continuos. Esa probabilidad está comprendida entre 0 y 1, siendo 0 la clase negativa y 1 la clase positiva.

Para ilustrar lo anterior, se propone el siguiente ejemplo. Supóngase que queremos determinar si un determinado billete es falso o auténtico, en función de los datos recolectados al realizar una fotografía de alta calidad. Para simplificar este ejemplo, supóngase que todos los parámetros de la fotografía se han resumido en un único indicador X (por ejemplo, la varianza de la imagen). Definimos como clase positiva ($y = 1$) el billete falso y como clase negativa ($y = 0$) el billete auténtico. Nuestro objetivo es estimar la probabilidad de que el billete sea falso en función de los parámetros recogidos por la cámara, matemáticamente $P(X) = P(Y = 1|X : \beta)$, definido como la probabilidad de que $y = 1$ dado una X parametrizada por β . Para representar esto podríamos emplear una regresión lineal de la siguiente forma:

$$P(X) = \beta_0 + \beta_1 X.$$

El problema de la regresión logística es que esta no se encuentra acotada, y por lo tanto podríamos predecir valores de $P(X) < 0$ o $P(X) > 1$, lo cual para este tipo de problemas no sería adecuado. Para paliar este problema se puede modelar $P(X)$ usando una función cuya salida esté acotada entre 0 y 1 para todos los valores de X . En regresión logística se emplea la función sigmoide (también denominada función logística):

$$P(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

Gracias a esta función se puede definir un umbral de pertenencia a la clase, así por ejemplo si $P(X) \geq 0,5$ el ejemplo pertenece a la clase 1 y si $P(X) < 0,5$ pertenece a la clase 0 [JWHT13, TSK16]. La figura 10.6 ilustra este ejemplo.

Como se observa en la función logística, los parámetros β_0 y β_1 son desconocidos y deben ser estimados basándose en los datos de entrenamiento disponibles. Para ello se emplea una estimación máximo verosímil cuya idea subyacente es la siguiente: se trata de buscar los valores de $\hat{\beta}_0$ y $\hat{\beta}_1$ tal que al introducirlos en la función logística se obtenga un número cercano a uno para todos los billetes falsos y un número cercano a 0 para aquellos que son auténticos.

Una vez se ha realizado el ajuste del modelo (entrenamiento) y se dispone de los coeficientes β del modelo, se puede utilizar este modelo para realizar predicciones sobre nuevas instancias (nuevos billetes para los cuales se quiere determinar su autenticidad). Téngase en cuenta que la regresión logística siempre devuelve una

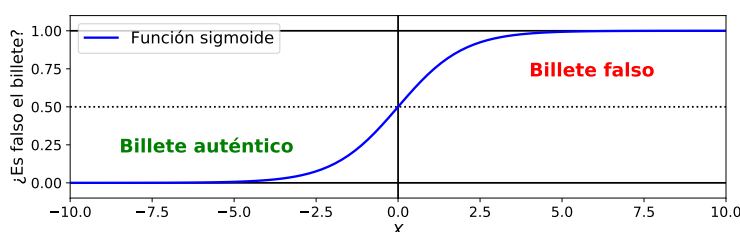


Figura 10.6: Ejemplo de regresión logística para clasificar billetes falsos en función de un parámetro X .

probabilidad por clase. La clase predicha será por tanto aquella con mayor probabilidad. Cuando el problema presenta más de dos clases, lo habitual es ofrecer las probabilidades normalizadas para que la suma de las probabilidades de todas las clases para cada instancia sea igual al 100 %.

Téngase en cuenta también que es posible proporcionar más parámetros de entrada. Por ejemplo, podríamos tener un vector de entrada $(X_1, X_2, \dots, X_k)$ con diferentes parámetros de la fotografía. En este caso sería necesario estimar los parámetros $\beta_0, \beta_1, \dots, \beta_k$. Por definición, los datos de entrada $(X_1, X_2, \dots, X_k)$ en la fórmula de regresión logística deben ser valores numéricos. Sin embargo, es posible utilizar cualquier otro tipo de atributos, aplicando previamente un conjunto de transformaciones a los campos categóricos, de texto y de elementos, de forma que estos se transformen en atributos numéricos.

10.4.2. Máquinas de Vectores de Soporte

Las **Máquinas de Vectores de Soporte** o *Support Vector Machines* (SVM) es un modelo de aprendizaje automático muy potente y versátil, capaz de realizar una clasificación lineal o no lineal, una regresión e incluso la detección de valores atípicos. Este tipo de algoritmos aparecieron en 1992, aunque sus fundamentos se remontan a los años 60 [HPK11].

Las SVM han demostrado tener un gran rendimiento cuando son aplicadas a la clasificación de conjuntos de datos complejos pero de tamaño pequeño o mediano, por lo que se las considera uno de los mejores clasificadores de «caja negra» (no son interpretables). Por este motivo, las SVM se han convertido uno de los modelos más populares en el aprendizaje automático, y cualquier persona interesada en el aprendizaje automático debe tenerlas en cuenta a la hora de realizar un análisis de datos.

La mejor manera de explicar el funcionamiento de las SVM es mediante algunas imágenes. La figura 10.7 muestra un hipotético conjunto de datos sobre el ejemplo anterior (determinar la autenticidad de un billete). En este caso se consideran dos parámetros X_1 y X_2 , lo que nos permite representar cada observación (billete del conjunto de entrenamiento) en dos dimensiones. Los cuadrados azules representan los billetes falsos, mientras que los círculos de color verde oliva representan los billetes auténticos.

Obsérvese en esta figura que las dos clases pueden separarse fácilmente con una línea recta (son linealmente separables). La figura de la izquierda muestra los límites de decisión de tres posibles clasificadores lineales. El modelo cuyo límite de decisión está representado por la línea discontinua de color verde es tan malo que ni siquiera separa las clases correctamente. Los otros dos modelos (representados de color rojo y morado) funcionan perfectamente en este conjunto de entrenamiento, pero sus límites de decisión se acercan tanto a las observaciones que probablemente no funcionarán bien en las nuevas observaciones (nuevos billetes a clasificar). Por el contrario, la línea sólida de color negro en el gráfico de la derecha representa el límite de decisión de un clasificador SVM; esta línea no sólo separa las dos clases, sino que también se mantiene lo más lejos posible de los elementos más cercanos. Un clasificador SVM intentará dejar el margen más ancho posible (representada por las líneas discontinuas paralelas) entre las clases. Esto se denomina **clasificador de margen máximo**.

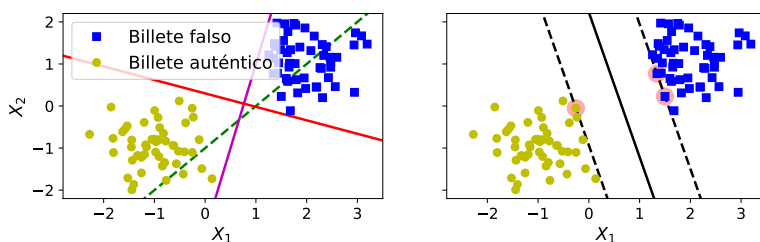


Figura 10.7: Ejemplo de SVM para clasificar billetes falsos en función de los parámetros X_1 y X_2 .

Cuando se extiende este concepto a más de dos dimensiones, la línea que separa las distintas clases se denomina **hiperplano** y los valores de cada clase más próximos a este hiperplano, y que por tanto permiten definirlo, se denominan **vectores de soporte**. Nótese que los vectores de soporte de ambas clases son equidistantes con respecto al hiperplano que los separa.

Como se puede imaginar el lector, en muchas ocasiones no es posible separar el conjunto de entrenamiento en dos clases de una forma tan sencilla, porque no existe un hiperplano de separación. Pero se puede extender el concepto de hiperplano para que este separe las clases de manera casi completa, llamado de margen suave. Esta generalización del caso claramente separable a uno no separable mediante un hiperplano de margen máximo se conoce como **clasificador de soporte vectorial** o *Support Vector Classifier* en inglés.

En este algoritmo se permite no solo que algunas observaciones estén dentro del margen, si no que también estén en lado incorrecto del hiperplano (estas últimas estarían mal clasificadas). Esto le da al algoritmo una mayor robustez frente al sobreaprendizaje, es decir, dotará al modelo de mayor capacidad de generalización y que de esta manera pueda clasificar mejor las nuevas observaciones.

Como se ha visto, esta técnica nos permite clasificar clases separadas linealmente. Pero, ¿qué pasa cuando estas clases no son separables linealmente? En este caso es posible realizar una transformación de los datos, proyectándolos en un espacio de mayores dimensiones donde los datos si estén separados linealmente por un hiperplano, y entonces se aplica el algoritmo SVM en él. Este espacio se denomina **espacio de características** y a las funciones que permiten realizar esta transformación se las denomina **funciones núcleo** o *kernel*. No obstante, esto queda fuera del alcance de este libro.

10.4.3. Árboles de decisión

Los **árboles de decisión** son una técnica de aprendizaje automático muy versátil que nos permite resolver tanto problemas de clasificación como de regresión. Son algoritmos muy potentes, capaces de ajustar conjuntos de datos complejos mediante el aprendizaje de reglas de decisión sencillas inferidas a partir de las características de los datos. Como veremos más adelante, los árboles de decisión son también los componentes fundamentales del que están compuestos otros algoritmos más complejos como los Random Forests, los cuales encuentran entre los algoritmos de aprendizaje automático más potentes y más utilizados de la actualidad.

Los árboles de decisión se construyen a base de ir haciendo preguntas sobre características determinadas a las observaciones del conjunto de datos utilizado. En base a las respuestas proporcionadas a cada pregunta realizada, se continúan realizando nuevas preguntas, hasta llegar un momento en el cual el algoritmo es capaz de clasificar estas observaciones según las respuestas a estas preguntas. Se sigue por lo tanto un esquema de preguntas que queda organizado jerárquicamente en forma de árbol, es decir, en forma de grafo dirigido compuesto por nodos (las preguntas) y aristas (las respuestas) [Qui96]. Por tanto un árbol de decisión trabaja como un “clasificador”. Las diferentes opciones de clasificación (respuesta a las preguntas) son excluyentes entre sí, lo que hace que a partir de nuevas observaciones desconocidas para el algoritmo, y siguiendo el árbol adecuadamente, se llegue a una única conclusión o decisión a tomar (la clase asignada a esta nueva observación).

Imagínese que queremos entrenar un árbol de decisión para determinar si se debe jugar al Golf a partir de una serie de observaciones sobre los datos meteorológicos en un día concreto. La figura 10.8 muestra un pequeño ejemplo de un árbol de decisión para este problema. Suponga que nos proporcionan el parte meteorológico para mañana, y sabemos que el tiempo será soleado, con una baja humedad y viento. ¿Es recomendable jugar al Golf según el árbol de decisión de la figura 10.8?

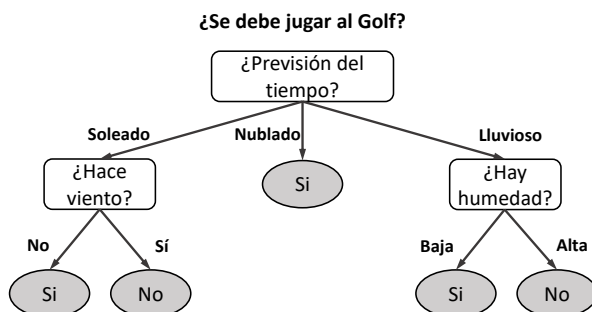


Figura 10.8: Ejemplo de un árbol de decisión para determinar si se debe jugar al Golf.

Los árboles de decisión presentan varias ventajas que los hacen una buena elección como método de clasificación o regresión. La primera de ellas es que son algoritmos simples y fáciles de interpretar, es decir, analizando el grafo resultante podemos entender su funcionamiento y en base a que decisiones se realizan las distintas perdiciones. También requieren poco preprocesado de los datos, ya que para aplicar esta técnica no es necesario normalizar los atributos para que tengan la misma escala. Por último, el coste de usar este modelo una vez entrenado es bastante bajo. Sin embargo, presentan algunas desventajas, como la tendencia al sobreajuste (impidiendo que puedan generalizar bien), que las perdiciones realizadas no son suaves ni continuas, la dificultad de entrenar un modelo que sea óptimo en un tiempo asumible o la tendencia de los árboles de decisión a crear modelos sesgados si algunas de las clases del modelo se encuentran sobrerrepresentadas.

10.4.4. Ensemble learning y Random Forests

Como hemos mencionado, uno de los inconvenientes de los árboles individuales es que pueden sobreaprender en el caso de hacerse demasiado complejos, es decir, sobreajustar sus datos. Esto hace que pese que su rendimiento sobre los datos de entrenamiento sería excelente, no serían capaces de generalizar bien a los datos nuevos. Para solucionar este problema aparecen los conjuntos de modelos.

Supongamos que planteamos una pregunta compleja sobre cierto tema a miles de personas elegidas al azar y luego agregamos sus respuestas. En muchos casos, la agregación de las respuestas es mejor que la respuesta de un experto en la materia. Esto se denomina comúnmente sabiduría de los grupos o inteligencia colectiva. Del mismo modo, si se agregan las predicciones de un grupo de modelos de predicción (ya sean modelos de clasificación o de regresión), a menudo se obtienen mejores predicciones que con el mejor modelo individual.

Un conjunto de modelos, o *ensemble* en inglés, es una colección de diferentes árboles de decisión individuales que se combinan para crear un modelo más sólido con un mejor rendimiento predictivo. Un conjunto de modelos individuales entrenados sobre un subconjunto de los datos del problema puede convertirse en un

buen predictor al promediar los errores de cada modelo individual. En general, los conjuntos realizan su tarea de clasificación o regresión mejor que un solo árbol de decisión porque son menos sensibles a los valores atípicos en sus datos de entrenamiento lo cual les ayuda a mitigar el riesgo de sobreajuste y generalizar mejor cuando se los aplica a nuevos datos. Además de lo anterior, debido a la estructura de estos modelos, estos modelos permiten la paralelización a la hora de entrenar y ejecutar los modelos. Por este motivo, estos algoritmos se convierten en decisiones muy acertadas cuando se trabajan con grandes conjuntos de datos.

La idea básica es remuestrear los datos y calcular las predicciones sobre el conjunto de datos remuestreados. Al promediar varios modelos conjuntamente se obtiene un mejor ajuste debido a que se reducen tanto los modelos con sesgo como los modelos con alta varianza. Si el algoritmo predice datos categóricos, entonces el voto de la mayoría dará la clase dominante o con mejor predicción. Si se está realizando predicción sobre datos numéricos, entonces se realizará la media sobre las predicciones. La figura 10.9 muestra la estructura de estos modelos.

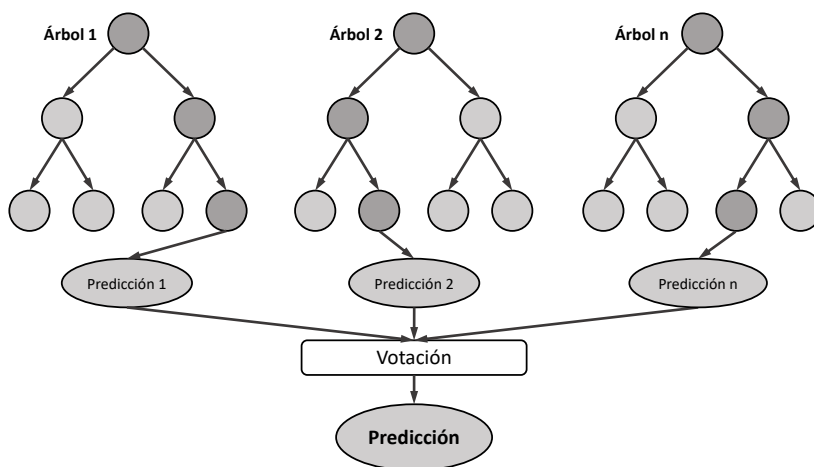


Figura 10.9: Estructura de un ensemble.



- **Sesgo (Bias):** Diferencia entre el valor proporcionado por el modelo y el valor real (error).
- **Varianza:** Cantidad de variación de la predicción del modelo según los datos que utilizamos se usen para entrenamiento.

Dentro de estas técnicas, destacamos algunos modelos que han alcanzado un alto grado de popularidad en los últimos años. El primero de ellos son los algoritmos de **bagging** que se basan en un enfoque que consiste en ajustar numerosos árboles de decisión, pero entrenados en diferentes subconjuntos aleatorios del con-

junto de entrenamiento. Este muestreo se realiza con reemplazo, es decir, se puede tomar varias veces la misma observación de entrenamiento. Cuando el muestreo se realiza sin reemplazo, el algoritmo se denomina *pasting*. Una vez entrenados todos los árboles, se puede hacer una predicción para una nueva instancia simplemente agregando las predicciones de todos los árboles individuales. La función de agregación suele ser tomar la predicción más frecuente para la clasificación, o la media para la regresión. En estos modelos, cada uno de los árboles individuales tienen un sesgo más alto que si se entrenara con el conjunto de entrenamiento original, pero la agregación reduce tanto el sesgo como la varianza. Generalmente, el resultado neto es que el conjunto tiene un sesgo similar pero una varianza más baja que un único árbol entrenado con el conjunto de entrenamiento original.

El segundo modelo son los denominados *Random Forests*. Estos modelos siguen una estrategia similar al Bagging pero en el momento de realizar el ajuste cada árbol elige un subconjunto aleatorio de las características del conjunto de entrenamiento, lo que añade más aleatoriedad. Esto da lugar a una mayor diversidad de árboles, que (una vez más) permite compensar un mayor sesgo por una menor varianza, lo que generalmente da lugar a un modelo globalmente mejor. De nuevo el resultado es el promedio de los árboles obtenidos. Los Random Forests son uno de los métodos más precisos de todos los métodos de clasificación.

Finalmente, el tercer modelo son los algoritmos denominados *Boosted Trees* o *Boosting*. La idea general de estos conjuntos de modelos es entrenar varios árboles de decisión que supone modelos débiles (*weak learners*), cada uno de los cuales intenta corregir los errores de su predecesor. Hay muchos métodos de Boosted Trees, pero los más populares son *AdaBoost* (abreviatura de Adaptive Boosting) y *Gradient Boosting*. Destacan principalmente algunas implementaciones de los métodos de Gradient Boosting como el algoritmo **XGBoost** [CG16], que se ha convertido en una herramienta muy utilizada en la industria y realmente popular entre los científicos de datos, ya que se ha probado eficaz en el ajuste de problemas de gran escala, como en el contexto del Big Data.

10.4.5. Redes neuronales

Finalmente, la última técnica de aprendizaje automático que describiremos serán las **redes neuronales**. Esta técnica es un modelo simplificado que basa su funcionamiento en las estructuras del cerebro humano y como este procesa la información. Las neuronas son los constituyentes estructurales del cerebro. Al cerebro siempre se le ha asociado tremenda capacidad de cálculo. Sus cualidades las alcanza mediante una sofisticada red de neuronas con interacciones sinápticas. Utilizando este símil, a mediados del siglo XX se publicaron los primeros estudios en los que se intenta replicar el comportamiento de las neuronas utilizando modelos matemáticos.

Cada neurona se conecta con otras a través de unos enlaces (tal y como haría en un cerebro humano). Cada uno de estos enlaces multiplica el valor de salida de la neurona anterior por un peso. Una red neuronal artificial es una estructura de varias capas de neuronas conectadas entre sí. Cada neurona está compuesta de

sinapsis (conexiones) con las neuronas de la capa anterior, asignando un peso a cada conexión; un sumador capaz de sumar las entradas anteriores; y finalmente una función de activación que limita la amplitud de la salida de la neurona dentro del intervalo $[0, 1]$.

Las redes neuronales artificiales son el núcleo del denominado aprendizaje profundo (o *deep learning* en inglés). Son versátiles, potentes y escalables, lo que las hace ideales para abordar tareas de aprendizaje automático grandes y muy complejas. Pongamos algunos ejemplos: La clasificación de miles de millones de imágenes (por ejemplo, el buscador de imágenes de google), los servicios de reconocimiento de voz (por ejemplo, el asistente virtual Siri de Apple), la recomendación de los mejores vídeos para ver a cientos de millones de usuarios cada día (por ejemplo, YouTube o Netflix), o el aprendizaje para vencer al campeón del mundo en Ajedrez (AlphaZero de DeepMind).

Un **perceptrón multicapa** (*Multilayer perceptron* en inglés) es un tipo de red neuronal artificial que está formada por múltiples capas. Este tipo de redes neuronales surgieron debido a las limitaciones de las arquitecturas más sencillas que no podían resolver problemas no lineales. En un problema de clasificación, un perceptrón multicapa se compone de una capa de entrada (con una neurona por cada atributo del conjunto de datos), una o más capas intermedias llamadas capas ocultas, y finalmente una capa de salida (con una neurona por cada clase/etiqueta del problema). Se dice que un perceptrón multicapa es un aproximador universal, ya que cualquier función continua en el espacio $\mathbb{R}^n$ puede aproximarse con un perceptrón multicapa con al menos una capa oculta de neuronas.

La figura 10.10 muestra un ejemplo de un perceptrón multicapa con una capa de entrada de 3 neuronas, dos capas ocultas de 4 neuronas y una capa de salida de 2 neuronas. Esta red neuronal podría aplicarse para ajustar un algoritmo de clasificación binaria (de dos clases) sobre un conjunto de datos con tres atributos.

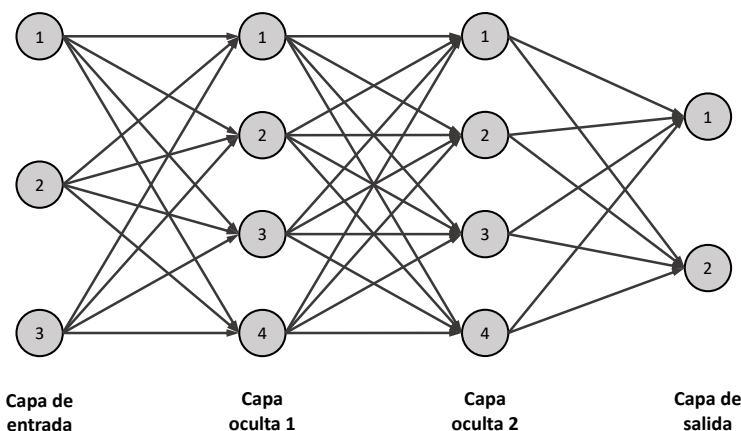


Figura 10.10: Estructura de un perceptrón multicapa con una capa de entrada de 3 neuronas, dos capas ocultas de 4 neuronas y una capa de salida de 2 neuronas.

Las neuronas de la capa de entrada no actúan como el resto de neuronas de la red, ya que únicamente se encargan de recibir las señales del exterior (características del conjunto de datos) y propagarlas al interior de la red. La última capa actuará como salida de la red, proporcionando la respuesta de esta hacia el exterior. Son por tanto las neuronas de las capas ocultas las que aproximan el comportamiento deseado. Es importante notar que en esta arquitectura las neuronas de cada capa están conectadas con las neuronas de la capa siguiente, sin bucles, por este motivo se las denomina también redes neuronales *feedforward*. Cuando las neuronas de una capa están completamente conectadas a todas las neuronas de la capa consecutiva, se dice que tenemos una red totalmente conectada. No obstante, se ha demostrado que se puede lograr un mejor rendimiento de las redes mediante la poda de algunas de estas conexiones durante el proceso de entrenamiento.

Debido su capacidad de aproximador universal, a su fácil uso y a su gran aplicabilidad en la industria, el perceptrón multicapa es actualmente una de las arquitecturas de redes neuronales más utilizadas, con numerosas implementaciones. Sin embargo, presenta algunas limitaciones, como la poca interpretabilidad de los modelos (son modelos de caja negra) y la gran cantidad de tiempo que puede llevar estimar el modelo cuando tiene un número elevado de neuronas (ya que crece enormemente el número de variables que debe estimar el modelo).